

Agent 工具调用主循环学习笔记

基于本次对话整理 - 工具表、block、WORKDIR、Path、协程与错误排查

重点: 本笔记的核心问题

模型到底什么时候调用工具?

block 里保存了什么?

TOOLS 和 TOOL_HANDLERS 分别给谁用?

Path.cwd().resolve() 和 .parent 怎么理解?

为什么 glob 工具会触发 KeyError: command?

目录

1. 本次对话总览
2. Agent 调用工具的本质
3. agent_loop 主循环逐段拆解
4. block、TOOLS、TOOL_HANDLERS 的分工
5. messages 如何保存上下文与工具结果
6. WORKDIR、Path 与安全路径
7. 点号 .、链式调用、管道类比
8. 逻辑父级 .parent 与 mkdir
9. 协程 coroutine 的含义
10. KeyError: command 的错误定位与修复
11. 最终复习清单

1. 本次对话总览

本次对话围绕一个最小 Coding Agent 的工具调用机制展开，重点不是让模型本身执行文件操作，而是理解“模型输出工具调用请求 - 外部 Runtime 执行函数 - 结果再回填给模型”的闭环。

主题	你提出的问题	最终结论
Agent 工具调用本质	是不是把可用工具表告诉模型，然后模型选择工具？	对。模型只输出 tool_use 请求，真正执行工具的是模型外部的 Agent Runtime。
文件工具函数	safe_path、run_read、run_write、run_edit、run_glob 每句怎么理解？	这些函数构成工具层，负责安全读写、编辑、搜索工作区文件。
WORKDIR 与 Path	WORKDIR、Path.cwd().resolve()、点号 . 怎么理解？	Path 是 pathlib 提供的路径对象；点号表示访问对象属性或调用对象方法。
逻辑父级与协程	.parent 是库定义的吗？什么是协程？	.parent 是 Path 的属性；协程是可暂停、等待、恢复的异步函数调用结果。
agent_loop	哪里真正调用工具函数？	handler(**block.input) 是真正执行工具的地方。
block 和 messages	block 是什么？工具结果是否已拿到？	block 是 SDK 解析后的内容块；工具结果已进入 output 并作为 tool_result 回填 messages。
错误排查	glob 后 KeyError: command 为什么？	调试打印写死了 block.input["command"]，但 glob 的参数是 pattern。

重点: 思维导图 - 学习路线

Agent 工具调用

|-- 概念层: 模型只负责选择工具和生成参数

|-- Runtime 层: 本地程序解析 tool_use 并执行函数

|-- 工具层: read/write/edit/glob 等 Python 函数

|-- 上下文层: messages 保存 user/assistant/tool_result

|-- 调试层: 根据 block.name 和 block.input 定位错误

2. Agent 调用工具的本质

Agent 工具调用的核心不是“模型真的执行工具”，而是“模型产生一个结构化调用请求”，然后由外部程序执行。

重点: 一句话记忆

大模型 = 决策大脑；Agent Runtime = 执行器；Tools = 可用能力；tool_use = 操作请求；tool_result = 执行反馈。

重点: 思维导图 - 工具调用闭环

用户请求

-> Agent Runtime 把 messages + TOOLS 发给模型

- > 模型判断是否需要工具
- > 如果需要: 输出 tool_use block
- > Runtime 根据 block.name 找真实函数
- > Runtime 执行 handler(**block.input)
- > 得到 output
- > 包装成 tool_result 放回 messages
- > 下一轮发给模型
- > 模型继续调用工具或给最终回答

因此，模型输出的是“工具调用意图 + 参数”，不是直接读文件、写文件、执行系统命令。

3. agent_loop 主循环逐段拆解

原始主循环代码

```
def agent_loop(messages: List):
    while True:
        response = client.messages.create(
            model=MODEL, system=SYSTEM, messages=messages,
            tools=TOOLS, max_tokens=8000,
        )
        messages.append({"role": "assistant", "content": response.content})

        if response.stop_reason != "tool_use":
            return

        results = []
        for block in response.content:
            if block.type == "tool_use":
                print(f"\033[33m> {block.name}\033[0m")
                handler = TOOL_HANDLERS.get(block.name)
                output = handler(**block.input) if handler else f"Unknown: {block.name}"
                print(str(output)[:200])
                results.append({"type": "tool_result", "tool_use_id": block.id, "content": output})

        messages.append({"role": "user", "content": results})
```

3.1 调模型的位置

```
response = client.messages.create(
    model=MODEL,
    system=SYSTEM,
    messages=messages,
    tools=TOOLS,
    max_tokens=8000,
)
```

这一步调用大模型。TOOLS 会作为工具说明书发给模型，模型据此决定要不要输出 tool_use。

3.2 保存模型输出

```
messages.append({"role": "assistant", "content": response.content})
```

这一步把模型刚才的回答保存到历史记录。注意：如果模型请求调用工具，这条 assistant 消息里保存的就是 tool_use block。

3.3 判断是否继续工具循环

```
if response.stop_reason != "tool_use":
    return
```

如果模型没有继续要求调用工具，循环结束。更完整的写法应该把最终文本返回出来，而不是直接 return None。

更适合交互程序的最终返回写法

```
if response.stop_reason != "tool_use":
    final_text = []
    for block in response.content:
        if block.type == "text":
            final_text.append(block.text)
    return "
".join(final_text)
```

3.4 真正调用工具函数的位置

重点: 最关键代码

真正执行工具函数的是: output = handler(**block.input)

```
handler = TOOL_HANDLERS.get(block.name)
output = handler(**block.input) if handler else f"Unknown: {block.name}"
```

第一行只是根据工具名找到函数对象；第二行才是执行函数。

变量	例子	含义
block.name	"read"	模型选择的工具名
TOOL_HANDLERS.get(block.name)	run_read	根据名字找到本地函数
block.input	{"path": "main.py", "limit": 100}	模型生成的工具参数字典
handler(**block.input)	run_read(path="main.py", limit=100)	真正执行工具函数
output	文件内容字符串	工具函数返回值

4. block、TOOLS、TOOL_HANDLERS 的分工

block 通常是 SDK 根据模型返回的 JSON 解析出来的对象。它不是普通字符串，而是已经分好属性的内容块。

对象	给谁用	作用	典型内容
TOOLS	模型	告诉模型有哪些工具、每个工具需要什么参数	name、description、input_schema
TOOL_HANDLERS	本地 Runtime	把工具名映射到真实 Python 函数	"read": run_read
block.name	Runtime	模型选择的工具名	"glob"
block.input	Runtime	模型生成的参数	{"pattern": "*.py"}
block.id	API 协议	将 tool_result 对应回这次 tool_use	toolu_xxx

重点: 思维导图 - TOOLS vs TOOL_HANDLERS

TOOLS - 发给模型看的说明书

- name: read / write / edit / glob
- description: 工具能做什么
- input_schema: 参数格式

TOOL_HANDLERS - 本地程序用的函数表

- "read" -> run_read
- "write" -> run_write
- "edit" -> run_edit
- "glob" -> run_glob

连接点

- 模型输出 block.name
- Runtime 用 block.name 查 TOOL_HANDLERS
- Runtime 执行 handler(**block.input)

重点: 容易混淆点

TOOLS 不负责执行工具，它只是工具 schema。真正执行工具依赖 TOOL_HANDLERS。

5. messages 如何保存上下文与工具结果

messages 是整个 Agent 的短期记忆，里面保存第一条用户请求、模型的 tool_use 请求、工具执行结果，以及后续模型回答。

重点: 思维导图 - messages 的增长过程

初始:

```
messages = [
  {role: "user", content: "Find all Python files"}
]
```

第 1 轮模型输出 tool_use:

```
messages += [  
    {role: "assistant", content: [ToolUseBlock(name="glob", input={"pattern":"*.py"})]}  
]
```

Runtime 执行工具后:

```
messages += [  
    {role: "user", content: [{type:"tool_result", tool_use_id:"...", content:"self_code.py  
main.py"}]}  
]
```

下一轮:

```
client.messages.create(messages=messages, tools=TOOLS)  
模型看到工具结果，再继续回答。
```

你之前说“怎么拿到工具调用返回结果还没有实现”，更准确地说：工具返回结果已经通过 output 拿到了，也已经作为 tool_result 放回 messages；缺的是最后一次无工具调用时，把最终文本 return 给外层调用者。

6. WORKDIR、Path 与安全路径

```
from pathlib import Path  
  
WORKDIR = Path.cwd().resolve()
```

Path 来自 Python 标准库 pathlib，用来以对象方式处理路径。WORKDIR 全大写表示按约定当作常量使用，但 Python 不会强制它不可修改。

重点: WORKDIR 的作用

WORKDIR 表示 Agent 允许操作的工作区根目录。所有 read/write/edit/glob 都应该被限制在 WORKDIR 内部。

6.1 safe_path 的安全边界

```
def safe_path(p: str) -> Path:  
    path = (WORKDIR / p).resolve()  
    if not path.is_relative_to(WORKDIR):  
        raise ValueError(f"Path escapes workspace: {p}")  
    return path
```

这个函数用来阻止路径逃逸。例如用户输入 ../secret.txt 时，resolve() 会把路径解析到工作区外，is_relative_to(WORKDIR) 会返回 False，于是抛出异常。

重点: 思维导图 - safe_path 安全路径检查

用户传入 p

-> WORKDIR / p 拼接路径
-> resolve() 解析成真实绝对路径
-> is_relative_to(WORKDIR) 检查是否仍在工作区内
|-- True: 返回安全 Path
|-- False: raise ValueError, 拒绝访问

6.2 Path 常用属性和方法

写法	类型	含义
Path.cwd()	类方法	获取当前工作目录
.resolve()	对象方法	把路径规范化为绝对真实路径
WORKDIR / p	运算符重载	路径拼接, 不是数学除法
.is_relative_to(WORKDIR)	对象方法	判断路径是否在 WORKDIR 内部
.read_text()	对象方法	读取文本文件内容
.write_text(content)	对象方法	写入文本, 默认覆盖原文件
.parent	属性	路径结构上的父级目录
.mkdir(parents=True, exist_ok=True)	对象方法	创建目录, 父目录不存在也一起创建

7. 点号 .、链式调用、管道类比

```
WORKDIR = Path.cwd().resolve()
```

点号 . 的核心作用是访问左侧对象的属性或方法。Path.cwd().resolve() 可以拆成两步：

```
p1 = Path.cwd()
p2 = p1.resolve()
WORKDIR = p2
```

重点: 点号不是通用管道

它有管道式的阅读感觉：上一步结果继续被下一步使用。但机制上，.resolve() 是 Path 对象自己的方法调用，左侧对象会作为 self 参与调用。

形式	例子	本质
链式调用	Path.cwd().resolve()	对 Path.cwd() 返回的对象调用 resolve 方法
函数嵌套	resolve(Path.cwd())	把 Path.cwd() 的结果作为普通函数参数
管道风格	Path.cwd() > resolve	某些语言中把前一步输出传给后一步函数, Python 标准语法没有这个管道

8. 逻辑父级 .parent 与 mkdir

```
file_path = Path("notes/python/day1.txt")
parent_dir = file_path.parent
# parent_dir 是 Path("notes/python")
```

.parent 来自 pathlib.Path。它表示路径结构上的父路径，不要求真实文件或目录已经存在。

重点: 逻辑父级

Path("a/b/c.txt").parent 会得到 Path("a/b")。即使 a/b/c.txt 这个文件不存在，Path 也能按路径文本结构算出父级。

```
file_path.parent.mkdir(parents=True, exist_ok=True)
```

参数	作用
parents=True	中间目录不存在时一起创建
exist_ok=True	目录已经存在时不报错

重点: 思维导图 - 写文件前创建父目录

```
run_write("notes/python/day1.txt", content)
-> safe_path 得到 /workspace/notes/python/day1.txt
-> file_path.parent 得到 /workspace/notes/python
-> mkdir(parents=True, exist_ok=True)
-> write_text(content) 写入文件
```

9. 协程 coroutine 的含义

协程是可以暂停、等待、再恢复执行的异步函数调用结果。普通函数调用后直接得到结果；async 函数调用后先得到 coroutine，需要 await 才能拿到最终结果。

普通函数

```
def normal_func():
    return 123

x = normal_func() # x 是 123
```

协程函数

```
async def async_func():
    return 123

x = async_func() # x 是 coroutine object, 不是 123
result = await async_func() # result 才是 123
```


重点: CoroutineType[Any, Any, Path]

如果 IDE 显示 WORKDIR 是 CoroutineType[Any, Any, Path]，意思是它现在是一个协程对象，await 后才会得到 Path。标准 pathlib.Path.cwd().resolve() 正常不应该是协程。

重点: 思维导图 - 协程理解模型

普通函数

|-- 调用: result = func()

|-- 结果: 立即得到返回值

|

协程函数 async def

|-- 调用: coro = async_func()

|-- 结果: 只创建协程对象

|-- 执行: result = await async_func()

|-- 用途: 网络请求、数据库、等待 I/O 等场景

10. KeyError: command 的错误定位与修复

运行现象

```
>>> Find all Python files in this directory
> glob
Traceback ...
KeyError: 'command'
```

这次模型选择了 glob 工具。glob 工具的参数通常是 pattern，而不是 command。你的调试打印代码写死了 block.input["command"]，所以当 input 里没有 command 时就崩溃。

问题代码

```
print(f"\033[33m$ {block.input['command']}\033[0m")
```

重点: 错误本质

不是 glob 工具坏了，而是打印逻辑假设所有工具都有 command 参数。read/write/edit/glob 的参数名各不相同。

工具名	典型 input	是否有 command
glob	{"pattern": "*.py"}	没有
read	{"path": "main.py", "limit": 100}	没有
write	{"path": "a.txt", "content": "..."}	没有
edit	{"path": "a.txt", "old_text": "...", "new_text": "..."}	没有

bash/run_command	{"command": "ls -la"}	有
------------------	-----------------------	---

10.1 推荐修复方式

通用打印方式

```
print(f"\033[33m> {block.name} {block.input}\033[0m")
```

这样无论工具参数是什么，都可以安全打印。

如果你想对 `command` 类工具特殊显示

```
if "command" in block.input:
    print(f"\033[33m$ {block.input['command']}\033[0m")
else:
    print(f"\033[33m> {block.name} {block.input}\033[0m")
```

10.2 更健壮的工具执行块

```
results = []

for block in response.content:
    if block.type == "tool_use":
        print(f"\033[33m> {block.name} {block.input}\033[0m")

        handler = TOOL_HANDLERS.get(block.name)

        if handler is None:
            output = f"Unknown tool: {block.name}"
        else:
            try:
                output = handler(**block.input)
            except Exception as e:
                output = f"Error running tool {block.name}: {e}"

        print(str(output)[:200])

        results.append({
            "type": "tool_result",
            "tool_use_id": block.id,
            "content": str(output),
        })

messages.append({"role": "user", "content": results})
```

重点: 思维导图 - `KeyError command` 排查流程

报错 `KeyError: 'command'`

-> 定位到 `block.input['command']`

-> 查看当前 block.name
-> 发现是 glob
-> 查看 glob 的 input schema
-> 参数是 pattern, 不是 command
-> 结论: 打印代码写死参数名
-> 修复: 打印 block.name + block.input 或使用 .get("command")

11. 最终复习清单

重点: 必须掌握的 8 句话

1. 模型不执行工具, 只输出 tool_use 请求。
2. TOOLS 是给模型看的工具 schema。
3. TOOL_HANDLERS 是给 Runtime 用的真实函数映射。
4. block 是 SDK 解析出来的内容块对象。
5. block.name 是模型选择的工具名。
6. block.input 是模型生成的参数字典。
7. handler(**block.input) 才是真正执行工具。
8. tool_result 要放回 messages, 让模型在下一轮看到工具结果。

问题	快速回答
哪里真正调用工具?	handler(**block.input)
TOOLS 是不是函数?	不是, 是工具说明书/schema
TOOL_HANDLERS 是什么?	工具名到 Python 函数的映射表
block 是 JSON 字符串吗?	底层可能来自 JSON, 但 SDK 中通常已解析为对象
工具结果在哪里?	output 变量, 然后包装为 tool_result
为什么 tool_result 是 user role?	这是 Messages API 的对话协议, 用用户消息把工具结果返回给模型
Path.cwd().resolve() 的点号是什么?	访问对象方法, 前一步返回对象继续调用自己的方法
.parent 是真实文件系统查询吗?	不是必须, 它是路径结构上的父路径属性
协程是什么?	async 函数调用产生的可等待对象, 需要 await 得到返回值
KeyError command 为什么?	当前工具 input 没有 command 参数

12. 一张总图串起来

重点: 思维导图 - 从用户输入到最终回答

用户: Find all Python files

- > messages 添加 user 消息
- > client.messages.create(messages, tools=TOOLS)
- > 模型输出 ToolUseBlock(name="glob", input={"pattern":"*.py"})
- > Runtime 保存 assistant tool_use 到 messages

```
-> Runtime 用 TOOL_HANDLERS["glob"] 找到 run_glob
-> 执行 run_glob(pattern="*.py")
-> output = "self_code.py
..."
-> Runtime 包装 tool_result 并追加到 messages
-> 下一轮 client.messages.create(...)
-> 模型读取 tool_result
-> 模型输出最终自然语言回答
-> agent_loop 返回/打印最终回答
```

学习建议：以后看任何 Agent 框架时，优先找三个位置：工具 schema 在哪里定义、工具 handler 在哪里注册、tool_result 如何回填到模型上下文。只要这三点看懂，就能看懂大多数工具调用主循环。